

Analisi del progetto

SayHi

1. Obiettivi del progetto

Realizzare un'applicazione Web capace di interagire con gli utenti tramite una chat (chatbot) utilizzando il linguaggio di programmazione JavaScript, inserito all'interno di una pagina HTML.

2. Analisi del Problema

Il problema principale è costruire un applicativo semplice da aprire, usare e validare, ma abbastanza completo da simulare un'esperienza simile a un chatbot moderno. La pagina HTML è il punto di accesso unico, mentre JavaScript gestisce interazioni, cronologia visiva e chiamate al modello AI.

2. Analisi del Problema

Requisiti individuati:

- Interfaccia chat chiara, con area messaggi e barra di input.
- Creazione, selezione, rinomina ed eliminazione delle conversazioni.
- Risposte generate da un modello AI remoto, non da risposte finte o precompilate.
- Design curato e responsive, con tema, animazioni e impostazioni base.
- Organizzazione ordinata dei file in cartelle dedicate.

3. Scelta del modello AI

Per la parte AI è stata scelta l'integrazione con Puter.js, che espone funzioni AI direttamente dal frontend tramite `puter.ai.chat()`.

Questa soluzione evita un backend locale e non richiede di inserire una API key personale nel codice sorgente.

3. Scelta del modello AI

Motivazione della scelta:

- Riduce la complessità tecnica rispetto a proxy o server dedicati.
- Permette di usare modelli in stile ChatGPT direttamente da JavaScript.
- Mantiene il progetto coerente con la consegna HTML + JavaScript.
- Consente domande aperte su argomenti diversi, superando il limite delle risposte hardcoded.

4. Architettura proposta

L'applicazione è organizzata in tre livelli semplici:

1. index.html: struttura della pagina e import di CSS, icone, Puter.js e script principale.
2. css/style.css: design liquid glass, layout responsive, sidebar, messaggi e modale impostazioni.
3. index.js: eventi, chat, input utente, rendering dei messaggi e chiamate AI.

Schema logico semplificato

Utente scrive un messaggio → JavaScript aggiunge la bolla utente → viene costruita la cronologia recente → Puter.js invia la richiesta al modello → la risposta viene mostrata con effetto streaming.

5. Bozza dell interfaccia

La UI è immaginata come una dashboard chat moderna: sidebar a sinistra con elenco conversazioni, area centrale per lo storico messaggi, header superiore con titolo chat e barra di input flottante in basso.

6. Strategia di test

- Apertura della pagina tramite Live Server o browser.
- Invio di domande semplici, come calcoli o richieste generali.
- Invio di domande contestuali, per verificare che la cronologia venga considerata.
- Test dei pulsanti: nuova chat, ricerca, copia, rigenera, impostazioni.
- Verifica dei casi di errore: mancato caricamento API, risposta vuota dal modello.

7. Limiti noti

Il funzionamento dell'AI dipende dalla disponibilità del servizio Puter.js e dalla connessione Internet. Se il servizio remoto non risponde, l'applicazione mostra un messaggio di errore comprensibile invece di bloccarsi.